ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

This is to certify that the project work entitled *"***FOODIEDELIGHT – Full-Stack Food Ordering Platform***"* is submitted by

DESINA ROHIT	**(24B11AI084)**
PILLI NAGENDRA	**(24B11AI344)**
CHAKRADHAR CHOWDARY GUNNAM	**(24B11AI054)**
BATTAGANI SURENDRA KUMAR	**(24B11AI031)**

in partial fulfillment of the requirements for award of the B.Tech. degree in Artificial Intelligence and Machine Learning.

Project Guide
Mrs.Annapurna,M.Tech(Ph.D)..
Assistant Professor
Department of AIML

Head of the Department
Dr. M. Venkatesh, M.Tech., Ph.D.
Assoc. Prof.
Department of AIML

DECLARATION

We hereby declare that the Cornerstone project entitled **"FOODIEDELIGHT – Full-Stack Food Ordering Platform"** is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech** degree in **Artificial Intelligence and Machine Learning**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place : Surampalem

Date:-

Submitted By

DESINA ROHIT	**(24B11AI084)**
PILLI NAGENDRA	**(24B11AI344)**
CHAKRADHAR CHOWDARY GUNNAM	**(24B11AI054)**
BATTAGANI SURENDRA KUMAR	**(24B11AI031)**

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs.Annapurna, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, Mrs. **Dr. R. Sunitha , Assistant Professor and Mr. A.Phani sridhar, Assistant Professor**, Department of AI&ML, for their support and valuable suggestions

Our deepest thanks to our Head of the Department **Dr. M. Venkatesh, Associate Professor**, for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean**, School of Engineering for their encouragement and support during the course of our project

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice -Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank **the faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

ABSTRACT

The Online Food Ordering System is a comprehensive full-stack web application developed using the MERN stack, which includes MongoDB for database management, Express.js and Node.js for backend server development, and React.js for building a dynamic and responsive user interface. The primary objective of this project is to provide a seamless, efficient, and secure digital platform that enables users to browse a wide variety of food items, view detailed menus with prices and categories, add selected items to a shopping cart, and place food orders online with ease.

The application supports three distinct user roles: Admins, Restaurant Partners, and standard Customers. The system eliminates manual ordering processes by automating menu management, order handling, and user interactions, thereby improving accuracy and efficiency. Users can create accounts, log in securely using JWT-based authentication, manage their personal profiles, track orders in real time, and view order history.

The platform features a premium glassmorphism UI design built with React.js (Vite), Framer Motion animations, and a multi-layered backend architecture. Restaurant partners manage their menus and fulfil incoming orders. Admins have complete control over the platform including food management, partner application approval, and order oversight. The system is scalable, reliable, and suitable for real-world food delivery services.

Keywords: MERN Stack, Online Food Ordering, MongoDB, React.js, Node.js, Express.js, E-commerce, Web Application, JWT Authentication, Cloudinary

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	Abstract	i
	Table of Contents	ii
	List of Figures	iii
	List of Tables	iii
1	Introduction	1
	1.1 Introduction	2
	1.2 Objectives of the Project	2
	1.3 Scope of the Project	2
2	System Overview	3
	2.1 System Architecture	3
	2.2 System Flow	3
3	Technologies Used	5
	3.1 Frontend Technologies	5
	3.2 Backend Technologies	6
	3.3 Database and Services	7
4	Database Design	8
	4.1 Database Collections	8
	4.2 Users Collection Schema	8
	4.3 Foods Collection Schema	8
	4.4 Orders Collection Schema	9
5	System Modules	10
6	Implementation	13
7	Output Screens	15
8	Conclusion	24
	References	26
	Appendix	27

LIST OF FIGURES

Figure No.	Name of the Figure	Page No.
7.1	Home Page – Main Landing Page	15
7.2	Login Page	16
7.3	Forgot Password Page	16
7.4	Register / Create Account Page	17
7.5	Menu Page – Food Browsing with Categories	17
7.6	Partner With Us – Restaurant Application Form	18
7.7	Cart – Your Cart Page	19
7.8	My Orders Page	19
7.9	Admin Dashboard – Orders Tab	20
7.10	Admin Dashboard – Add Food & Bulk Import	21
7.11	Admin Dashboard – Food List with Actions	21
7.12	Admin Dashboard – Partner Applications Tab	22
7.13	Admin Dashboard – Analytics Tab	23
7.14	User Profile Page	23

LIST OF TABLES

Table No.	Name of the Table	Page No.
2.1	System Architecture Layers	3
3.1	Technology Stack Summary	7
4.1	Users Collection Structure	8
4.2	Foods Collection Structure	8
5.1	Key API Endpoints	11

CHAPTER 1
INTRODUCTION

1.1 Introduction

FoodieDelight is a comprehensive, modern full-stack web application built using the MERN stack, which is a combination of four key technologies: MongoDB, Express.js, React.js, and Node.js. These technologies together enable the development of a complete web application covering both the frontend and backend layers. The platform provides a seamless, secure, and efficient food ordering experience for customers, restaurant partners, and administrators.

The application addresses the growing demand for digital food ordering solutions by providing a feature-rich platform that supports the entire food ordering lifecycle. From menu browsing and cart management to order placement, payment processing, and real-time tracking, FoodieDelight delivers a premium user experience powered by modern web technologies including a glassmorphism UI design and Framer Motion animations.

The system supports three distinct user roles. Customers browse menus, place orders, and track deliveries. Restaurant Partners manage their menus and fulfil incoming orders. Administrators oversee the entire platform, manage partner applications, and maintain system integrity. JWT-based authentication ensures secure access control across all three roles.

1.2 Objectives of the Project

The main objectives of the FoodieDelight platform are listed below.

1. To develop a complete full-stack web application using the MERN stack supporting three user roles: Customer, Restaurant Partner, and Admin.
2. To implement secure user authentication including email OTP verification, JWT-based sessions, and password reset via email links.
3. To build a modern, responsive menu browsing interface with Veg/Non-Veg filtering, category browsing, and dynamic food shuffling.

4. To provide a premium checkout experience with online payment simulation and Cash on Delivery support.
5. To allow customers to track orders in real time and cancel within a 5-minute grace period after placement.
6. To build a dedicated Restaurant Partner dashboard for menu management and smart order fulfilment with automatic COD payment synchronisation.
7. To build an Admin control panel for platform oversight, partner application management, global food control, and data export to Excel.
8. To integrate Cloudinary for secure, optimised, cloud-based food image hosting and CDN delivery.
9. To deploy the application on Vercel with proper environment configuration and legacy peer dependency handling.

1.3 Scope of the Project

The scope of the FoodieDelight platform covers the development of a fully functional web-based food ordering system with multi-role user management, real-time order tracking, payment integration, and admin oversight. The system handles the complete food ordering workflow from product browsing to delivery confirmation.

The frontend development uses React.js (Vite) to build all user interface pages, including the home page, authentication pages, menu page, cart, checkout, order tracking, restaurant partner dashboard, and admin panel. The backend uses Node.js and Express.js to build RESTful APIs for all operations. MongoDB stores all platform data via Mongoose ORM.

The project encompasses Cloudinary integration for food image management, SendGrid and Nodemailer for transactional emails, and CSV or Excel bulk import and export for admin operations. Deployment on Vercel with proper .npmrc configuration for legacy peer dependencies is also within scope. Features such as live GPS tracking, real payment gateway integration, and a native mobile application are identified as future enhancements.

CHAPTER 2
SYSTEM OVERVIEW

2.1 System Architecture

The FoodieDelight platform follows a three-tier architecture that separates the application into the Presentation Layer, the Application Logic Layer, and the Data Layer. This separation ensures that each layer has a clearly defined responsibility and can be developed and maintained independently of the others.

The Presentation Layer is built with React.js (Vite) and uses the Context API for global state management and Framer Motion for animations. The Application Logic Layer consists of Node.js and Express.js, which handle all API requests, authentication, business logic, and integration with external services such as Cloudinary and SendGrid. The Data Layer consists of MongoDB accessed via Mongoose ORM.

Table 2.1: System Architecture Layers

Layer	Technology	Responsibility
Presentation	React.js (Vite), Framer Motion	User Interface, State Management, API Calls
Application Logic	Node.js, Express.js	API Routing, Authentication, Business Logic
Data	MongoDB, Mongoose ORM	Data Storage, Schema Management, Queries
Media	Cloudinary, Multer	Image Upload and Cloud CDN Delivery
Email	Nodemailer, SendGrid	OTP Emails, Notifications, Alerts

2.2 System Flow

The flow of data within FoodieDelight follows a request-response pattern with role-based access control. When a customer places an order, the React frontend sends an authenticated HTTP request with a JWT token to the Express backend. The backend validates the token, processes the order, saves it to MongoDB, and returns a response that triggers the payment animation on the frontend.

For restaurant partners, when an order is received, the partner dashboard updates with the new order. The partner updates the status through Preparing, Out for Delivery, and Delivered stages. When a Cash on Delivery order is marked as Delivered, the system automatically sets the payment status to Paid and records the timestamp. Admin operations follow a similar request-response pattern, with rejection emails dispatched automatically when partner applications are declined.

CHAPTER 3
TECHNOLOGIES USED

3.1 Frontend Technologies

3.1.1 React.js (Vite)

React.js is an open-source JavaScript library developed by Meta for building user interfaces. It is based on a component-based architecture where each page and section is built as an independent, reusable component. Vite is used as the build tool, providing fast Hot Module Replacement and optimised production builds. React Router v6 handles client-side navigation with protected routes that check JWT token validity and user role.

3.1.2 Context API

The React Context API is used for global state management. The Auth Context manages user authentication state including the JWT token and user role, providing login, logout, and refresh functions. The Cart Context manages shopping cart items, quantities, and total price. This approach eliminates prop drilling across deeply nested components throughout the application.

3.1.3 Framer Motion

Framer Motion is a production-ready animation library for React. It is used throughout FoodieDelight for fluid micro-animations, page transitions, and the premium payment processing animation overlay with glassmorphism effects and pulsing loaders displayed during order placement.

3.1.4 React Icons and React Toastify

React Icons provides a comprehensive library of SVG icons used throughout the interface. React Toastify provides non-intrusive toast notification components for displaying success, error, and informational feedback messages to users across all actions on the platform.

3.2 Backend Technologies

3.2.1 Node.js

Node.js is a server-side JavaScript runtime environment built on Google Chrome's V8 JavaScript engine. It allows JavaScript code to be executed outside the browser, enabling full-stack development using a single language. Node.js powers the entire backend server of the FoodieDelight platform.

3.2.2 Express.js

Express.js is a fast, minimal, and flexible web application framework for Node.js. It is used to create all RESTful API endpoints for user authentication, menu management, cart operations, order processing, partner management, and admin operations. Express middleware handles CORS, JSON parsing, JWT verification, and file uploads.

3.2.3 JWT and Bcrypt.js

JSON Web Tokens are used for stateless, secure authentication across all API endpoints. Each authenticated request must include a valid JWT in the Authorization header. Bcrypt.js is used for password hashing, ensuring that plaintext passwords are never stored in the database.

3.2.4 Multer and Cloudinary

Multer is a Node.js middleware for handling multipart form data used for food image uploads. The multer-storage-cloudinary package connects Multer directly to Cloudinary, enabling optimised, cloud-based image storage and delivery via CDN for all food item images on the platform.

3.3 Database and Services

3.3.1 MongoDB and Mongoose

MongoDB is a NoSQL document database used to store all application data including users, foods, orders, and restaurant partner applications. Mongoose ORM provides schema-based modelling, data validation, and query building for all MongoDB collections used in the platform.

3.3.2 Email Services

Nodemailer is used for sending emails via SMTP using Gmail App Passwords. SendGrid provides an alternative transactional email service. These are used for OTP verification during registration, password reset links, and partner application approval or rejection notifications with the specific reason provided by the admin.

Table 3.1: Technology Stack Summary

Category	Technology	Purpose
Frontend Framework	React.js + Vite	UI Components and Build Tool
State Management	Context API	Auth and Cart State
Animations	Framer Motion	Micro-animations and Page Transitions
Backend Runtime	Node.js + Express.js	RESTful API Server
Authentication	JWT + Bcrypt.js	Token Auth and Password Hashing
Database	MongoDB + Mongoose	Data Storage and ORM
Media Storage	Cloudinary + Multer	Cloud Image Upload and CDN
Email	Nodemailer + SendGrid	OTPs and Notifications
Deployment	Vercel	Frontend and Backend Hosting

CHAPTER 4
DATABASE DESIGN

4.1 Database Collections

The FoodieDelight MongoDB database consists of five primary collections: users, foods, orders, carts, and restaurantapplications. Each collection is defined with a Mongoose schema that enforces data types, required fields, default values, and references between collections using ObjectId fields.

4.2 Users Collection Schema

Table 4.1: Users Collection Structure

Field	Type	Required	Description
_id	ObjectId	Auto	MongoDB unique identifier
name	String	Yes	Full name of the user
email	String (unique)	Yes	Email address – must be unique
password	String	Yes	Bcrypt hashed password
role	String (enum)	Yes	user / restaurant / admin
isVerified	Boolean	No	Email OTP verified status
otp	String	No	Temporary OTP for verification
otpExpiry	Date	No	OTP expiration timestamp
resetToken	String	No	Password reset token
createdAt	Date	Auto	Account creation timestamp

4.3 Foods Collection Schema

Table 4.2: Foods Collection Structure

Field	Type	Required	Description
_id	ObjectId	Auto	MongoDB unique identifier
name	String	Yes	Food item name
description	String	Yes	Food item description
price	Number	Yes	Price of the food item
category	String	Yes	Food category (e.g., Biryani)

isVeg	Boolean	Yes	Vegetarian indicator flag
imageUrl	String	Yes	Cloudinary image URL
isAvailable	Boolean	No	Availability toggle – default true
restaurantId	ObjectId (ref)	Yes	Reference to restaurant owner
restaurantName	String	Yes	Name of the restaurant

4.4 Orders Collection Schema

The Orders collection stores all order data including the items ordered, customer details, payment information, and order status. Orders reference the user ID and contain an embedded array of food items with quantities and prices at the time of ordering. This snapshot approach ensures order history accuracy even if food items are later edited or deleted by restaurant partners.

The paymentDetails subdocument captures the payment method (Online or Cash on Delivery), the payment status (Paid or Pending), and the paidAt timestamp which is recorded automatically when a restaurant partner marks a COD order as Delivered through the Auto-Payment Sync feature.

CHAPTER 5
SYSTEM MODULES

5.1 Customer Module

The Customer Module provides all features available to registered customers. Authentication begins with email OTP verification during registration, standard email and password login, and a Forgot Password workflow that sends reset links to the user's email address. JWT tokens are issued upon successful login and are used to secure all subsequent API requests made by the customer.

The menu browsing interface features a glassmorphism design with floating animations powered by Framer Motion. Customers can filter food items by Veg or Non-Veg preference and browse by categories. A dynamic food shuffling feature presents items in varying orders to improve discovery. Each food card shows the image, name, description, price, and an Add to Cart button.

The cart page shows all added items with quantities and totals, and a Proceed to Checkout button. Customers choose between Online Payment simulation and Cash on Delivery. The My Orders page shows all past and current orders with the restaurant name, expandable payment details, and a cancel button available within a 5-minute grace period.

5.2 Restaurant Partner Module

The Restaurant Partner Module enables food businesses to operate on the FoodieDelight platform. Potential partners submit an application via the Partner With Us form with business details and supporting documents. Upon admin approval, the restaurant gains access to a dedicated partner dashboard with full menu and order management capabilities.

The Menu Control feature allows partners to add food items with Cloudinary image uploads, edit existing items, delete items, and toggle availability using the Mark Unavailable feature. Unavailable items are hidden from customers. The Smart Order

Fulfillment system allows partners to update order status through the Preparing, Out for Delivery, and Delivered stages, with Auto-Payment Sync for COD orders.

5.3 Admin Module

The Admin Module provides a Global Dashboard as a supreme control panel for the entire platform. The Application Management feature allows admins to review, approve, or reject restaurant partner applications. Automated rejection emails are sent to applicants containing the specific reason provided by the admin through the Partner Applications tab.

The Global Food Control feature shows all foods across all restaurants with filters by category or restaurant owner. Admins can edit, delete, or Mark Unavailable any food item, and bulk import food data using CSV or Excel files. The Analytics tab provides a global sales overview with revenue trends and order status breakdowns. Order data can also be exported to Excel for reporting.

Table 5.1: Key API Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register with OTP email verification
POST	/api/auth/login	Public	Login and receive JWT token
POST	/api/auth/forgot-password	Public	Send password reset email link
GET	/api/foods	Public	Get all available food items
POST	/api/foods	Restaurant	Add new food item with image
PUT	/api/foods/:id	Restaurant	Edit food item details
PATCH	/api/foods/:id/availability	Restaurant/Admin	Toggle food item availability
POST	/api/orders	Customer	Place a new food order

GET	/api/orders/my	Customer	Get current customer orders
PATCH	/api/orders/:id/cancel	Customer	Cancel order within 5 minutes
PATCH	/api/orders/:id/status	Restaurant	Update status with auto COD sync
GET	/api/admin/applications	Admin	Get all partner applications
PATCH	/api/admin/applications/:id	Admin	Approve or reject application

CHAPTER 6
IMPLEMENTATION

6.1 Frontend Implementation

The frontend of FoodieDelight is built with React.js using Vite as the build tool. The project structure organises components by user role and feature, with separate directories for customer pages, restaurant partner pages, and admin pages. React Router v6 handles client-side navigation with protected routes that check JWT token validity and user role before granting access to restricted pages.

Global state is managed using two Context Providers. The AuthContext stores the authenticated user object, JWT token, and role, providing login, logout, and token refresh functions consumed throughout the application. The CartContext manages cart items, quantities, and total price, with functions for adding, removing, and updating items in the cart.

The glassmorphism design system is implemented using CSS backdrop-filter, semi-transparent backgrounds, and subtle border gradients. API calls are made using the browser fetch API directed to the VITE_API_URL environment variable. All authenticated requests include a Bearer token in the Authorization header extracted from the AuthContext.

6.2 Backend Implementation

The backend server is implemented with Node.js and Express.js using a modular architecture with separate route files, controller files, middleware, and model files. The server applies CORS middleware to allow cross-origin requests from the React frontend and express.json() middleware for parsing request bodies.

JWT authentication is implemented as Express middleware. The authMiddleware function extracts the token from the Authorization header, verifies it using the JWT_SECRET environment variable, and attaches the decoded user object to the request. Role-based access is enforced by additional requireAdmin and requireRestaurant middleware functions applied to protected routes.

Email functionality is implemented using Nodemailer with Gmail SMTP. OTP emails are generated with a 6-digit code stored temporarily in the user document with a 10-minute expiry. Password reset emails contain a signed JWT reset token embedded in a link. Partner rejection emails are composed dynamically with the admin-provided rejection reason included in the message body.

6.3 Database Implementation

The MongoDB database is connected using Mongoose at application startup with the MONGO_URI from environment variables. Five Mongoose schemas are defined for User, Food, Order, Cart, and RestaurantApplication. Each schema includes field-level validation, required constraints, default values, and ref fields for population queries.

The Order schema uses an embedded array of order items rather than document references, to preserve a snapshot of the food details at the time of ordering. This ensures order history accuracy even if food items are later edited or deleted. The paymentDetails subdocument captures the method, status, and paidAt timestamp for complete payment audit records.

6.4 Deployment Implementation

The application is deployed on Vercel. Because npm v7 and above enforces strict peer dependency resolution, and the multer-storage-cloudinary package requires older package versions, the backend directory includes an .npmrc file with legacy-peer-deps set to true. This ensures Vercel successfully builds the Node.js environment without dependency conflicts during the deployment process.

Environment variables are configured in the Vercel dashboard, including MONGO_URI, JWT_SECRET, Cloudinary credentials, and SMTP configuration. The frontend VITE_API_URL is set to the Vercel backend deployment URL. Both frontend and backend are deployed as separate Vercel projects with the appropriate root directory settings configured in each project.

CHAPTER 7
OUTPUT SCREENS

7.1 Home Page

The home page is the main landing page of FoodieDelight. It features a full-width hero section with the tagline 'Savor Every Bite, Delivered to Your Door' displayed prominently over a rich food photography background. The navigation bar includes the FoodieDelight logo, a search bar, and links for Menu, Partner With Us, My Orders, and a cart icon showing the current item count.

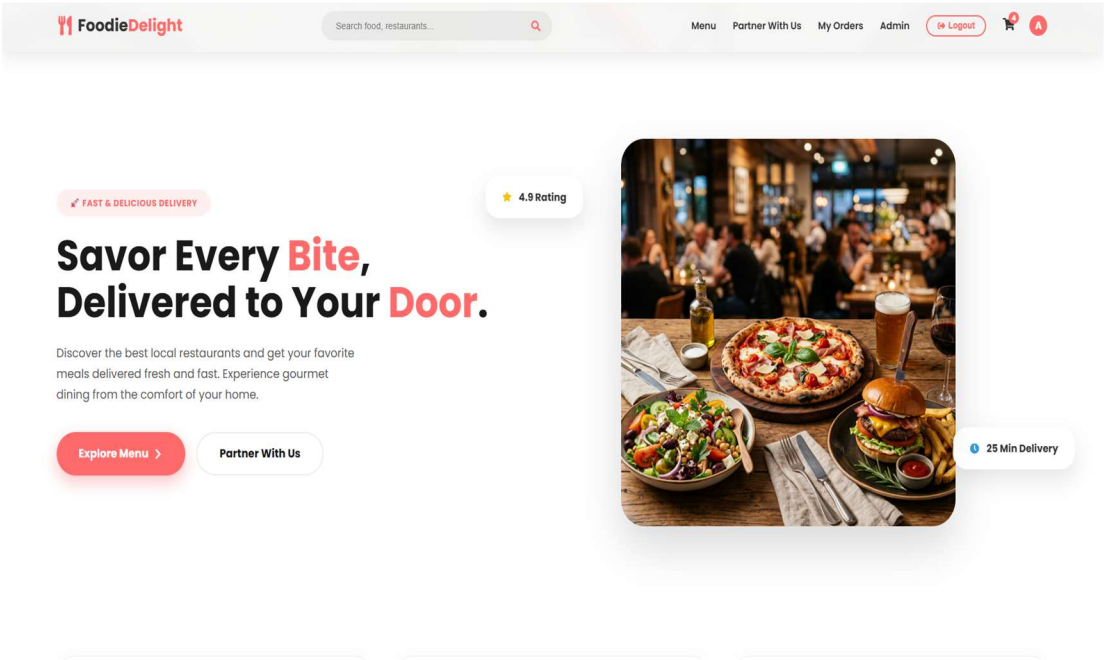

Figure 7.1: Home Page – Main Landing Page

7.2 Login Page

The login page presents a glassmorphism 'Welcome Back' card centred over the food photography background. The card contains input fields for Email and Password, a Login button, and a link to the Sign Up page. Incorrect credentials display a specific toast notification error message to the user.

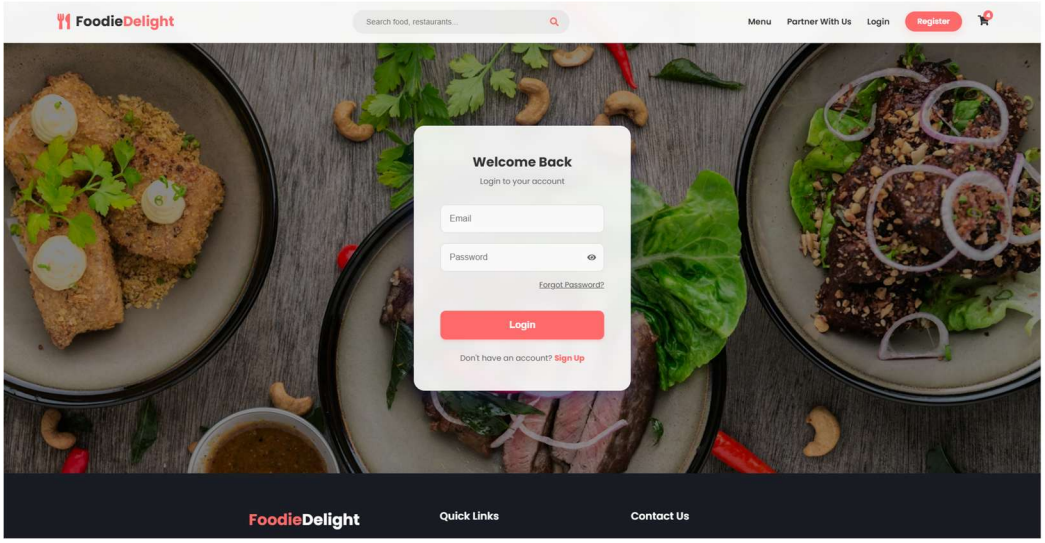

Figure 7.2: Login Page

7.3 Forgot Password Page

The Forgot Password page allows users who cannot remember their password to request a reset link. Users enter their registered email address and submit the form. The system generates a signed JWT reset token, embeds it in a reset link, and sends it to the user's email address via the configured SMTP email service.

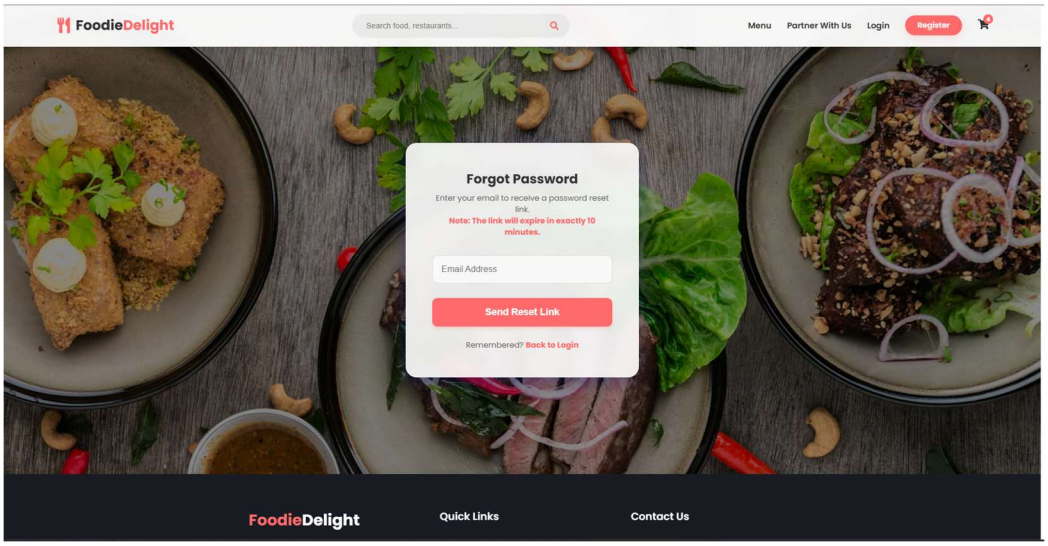

Figure 7.3: Forgot Password Page

7.4 Register Page

The registration page presents a Create Account glassmorphism card with fields for Full Name, Email, and Password, followed by a Continue button. After submission, a 6-digit OTP is sent to the registered email address and the user is directed to an OTP verification step before their account is fully activated on the platform.

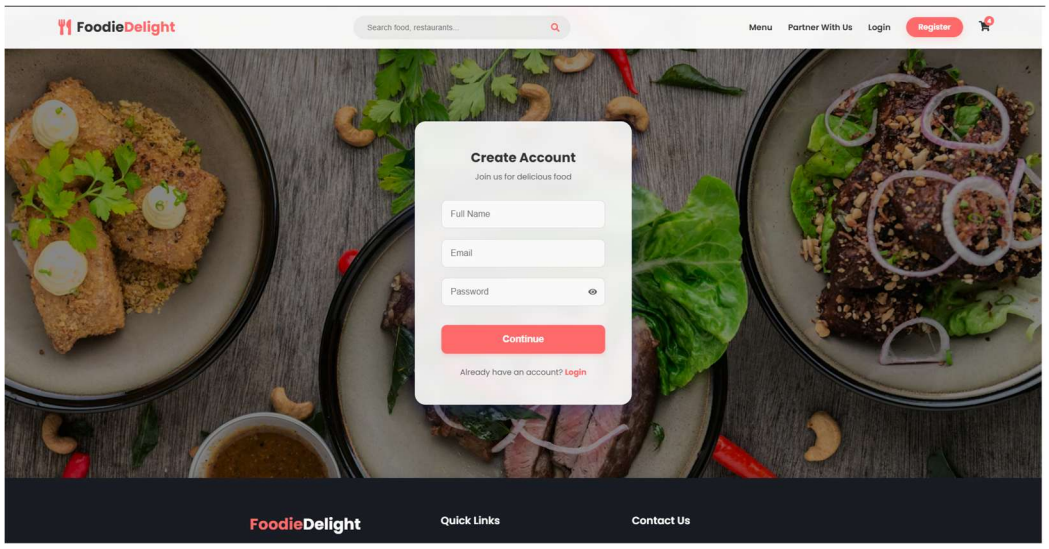

Figure 7.4: Register / Create Account Page

7.5 Menu Page

The menu page displays all available food items in a responsive card grid. A category sidebar allows customers to filter items by food type such as Biryani, Starters, and Desserts. Each food card shows the item image, name, price, restaurant name, and an Add to Cart button. The Veg and Non-Veg filter toggles are shown at the top for quick dietary filtering.

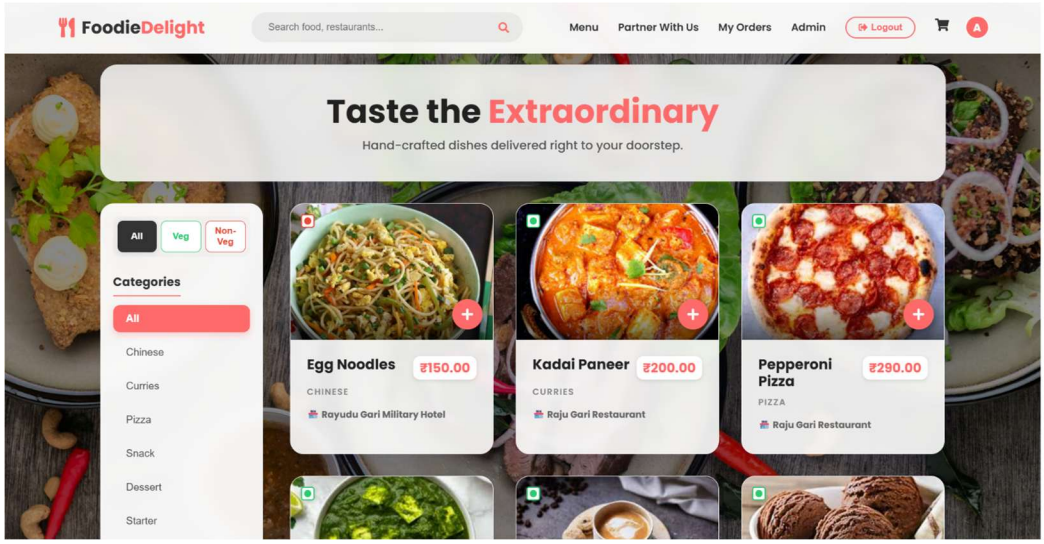

Figure 7.5: Menu Page – Food Browsing with Categories

7.6 Partner with Us Page

The Partner with Us page presents a restaurant application form where potential partners submit their Restaurant Name, Owner's Full Name, Email Address, Phone Number, and a single PDF document containing their Menu, FSSAI Government Certificate, and Restaurant Address. Submitted applications are reviewed by the admin through the Partner Applications tab.

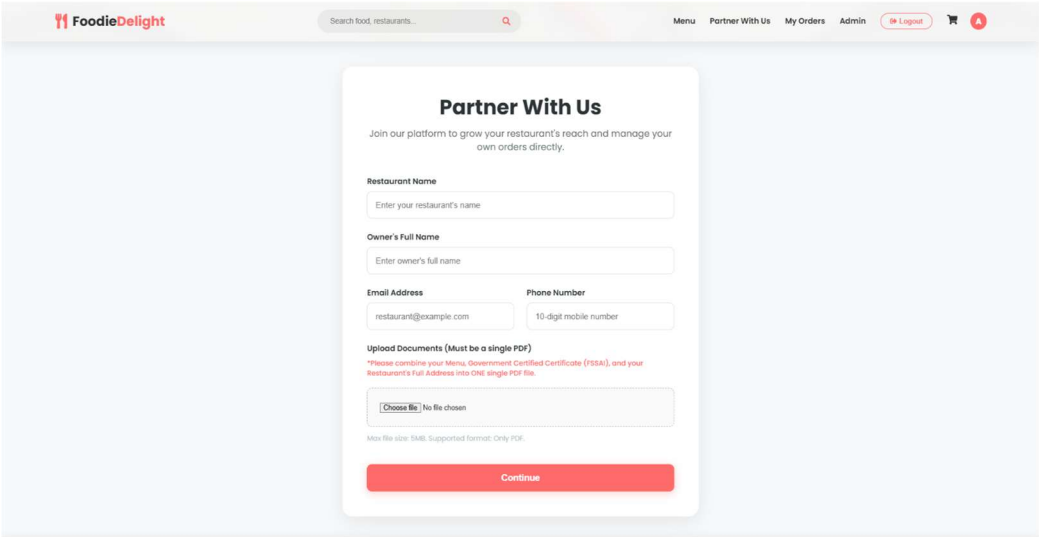

Figure 7.6: Partner With Us – Restaurant Application Form

7.7 Cart Page

The cart page displays all food items added by the customer, with item names, individual prices, quantities, and a computed total. A Delivery Address section shows the saved address with a Change option. The Order Summary panel on the right shows the subtotal, delivery fee, and final total, with a Proceed to Checkout button to initiate the order.

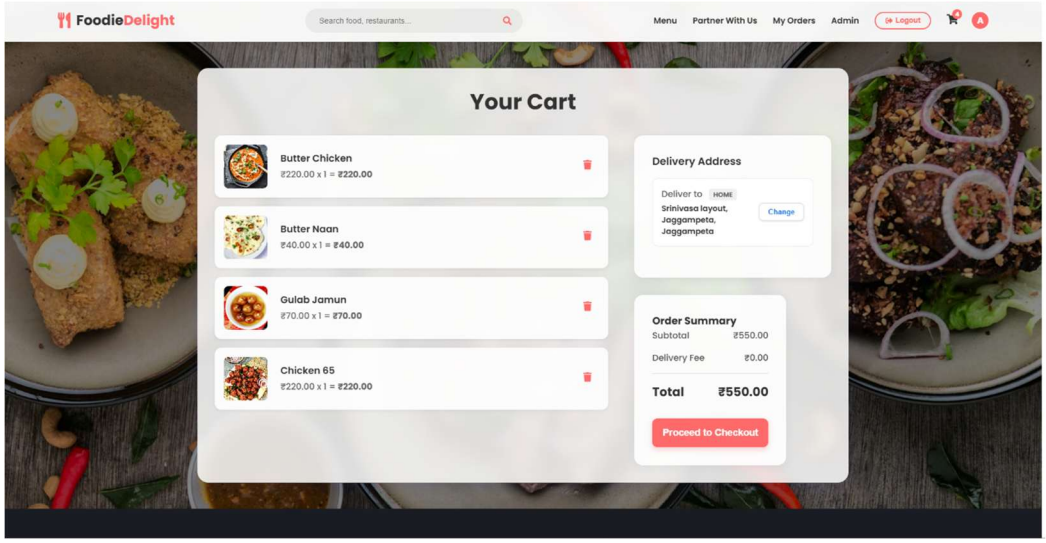

Figure 7.7: Cart – Your Cart Page

7.8 My Orders Page

The My Orders page displays all past and current orders for the logged-in customer. Each order card prominently shows the Restaurant name, the order ID, date, and total amount with the current delivery status badge. Orders with a Delivered status are shown in green. Customers can expand individual order cards to view the full payment details and items.

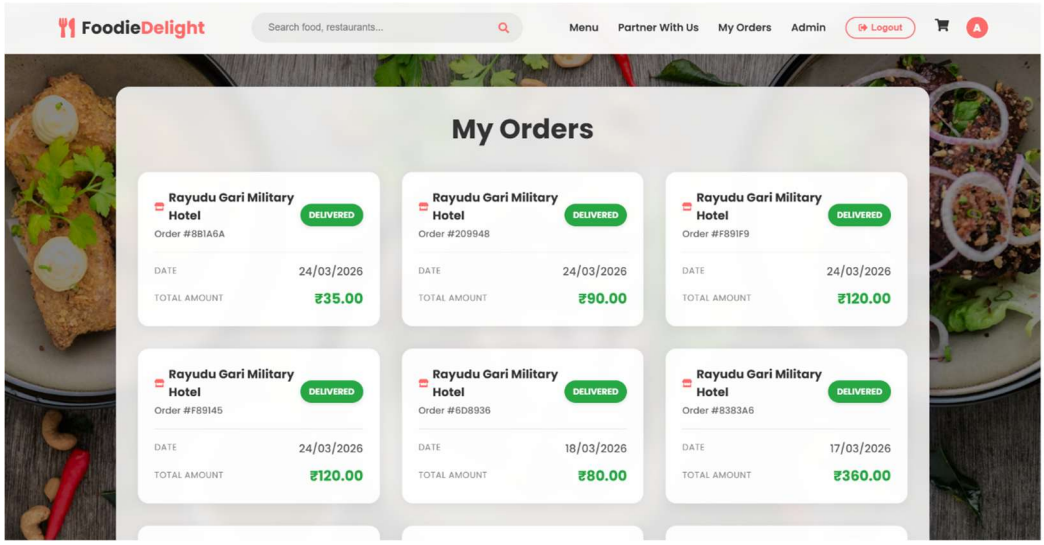

Figure 7.8: My Orders Page

7.9 Admin Dashboard – Orders Tab

The Admin Dashboard Orders tab provides a complete Active Orders Overview table showing all platform orders. Each row displays the Order ID, Date, Customer name, Total amount, Payment method with Paid or Unpaid status, current delivery status badge, and an action dropdown to manually update the order status. An Export Orders button allows order data download in Excel format.

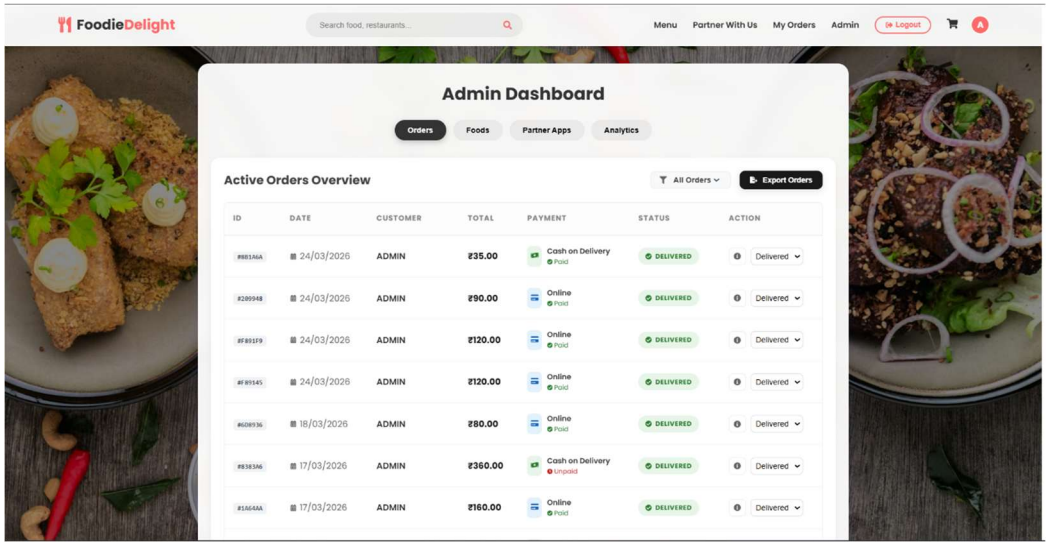

Figure 7.9: Admin Dashboard – Orders Tab

7.10 Admin Dashboard – Add Food and Bulk Import

The Admin Dashboard Foods tab provides an Add New Food form on the left side for adding individual food items with Name, Category, Price, Veg/Non-Veg dropdown, Image URL, and Description fields. On the right side, the Bulk Food Import panel allows admins to upload a CSV or Excel file and import multiple food items at once, with supported column details shown below the upload button.

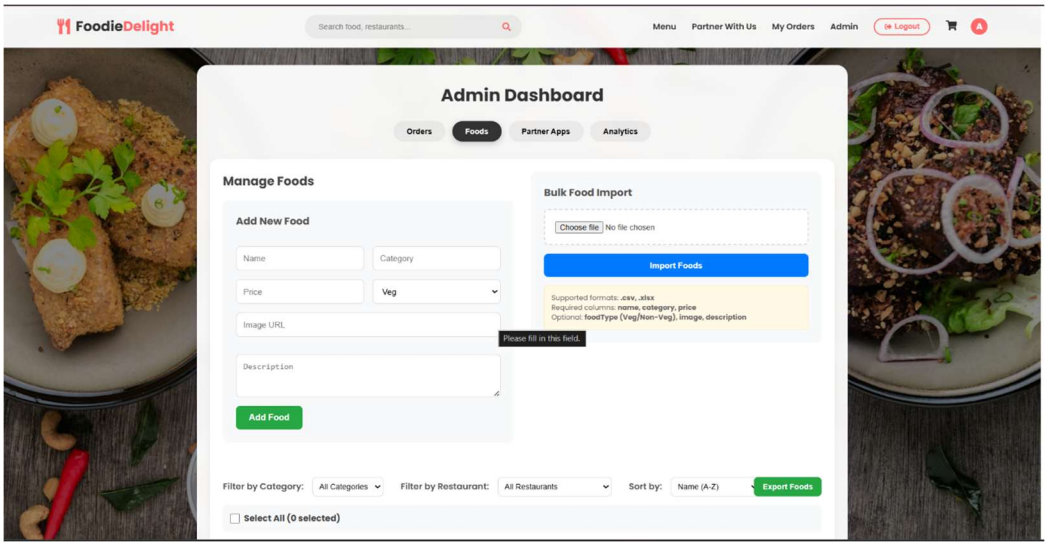

Figure 7.10: Admin Dashboard – Add Food and Bulk Import

7.11 Admin Dashboard – Food List with Actions

The lower section of the Foods tab displays a complete list of all food items across all restaurants on the platform. Each item row shows the food image, name, Veg or Non-Veg badge, price, and the owning restaurant name. Action buttons for Mark Unavailable, Edit, and Delete are provided per item. Filters by Category and by Restaurant are available at the top, along with an Export Foods button.

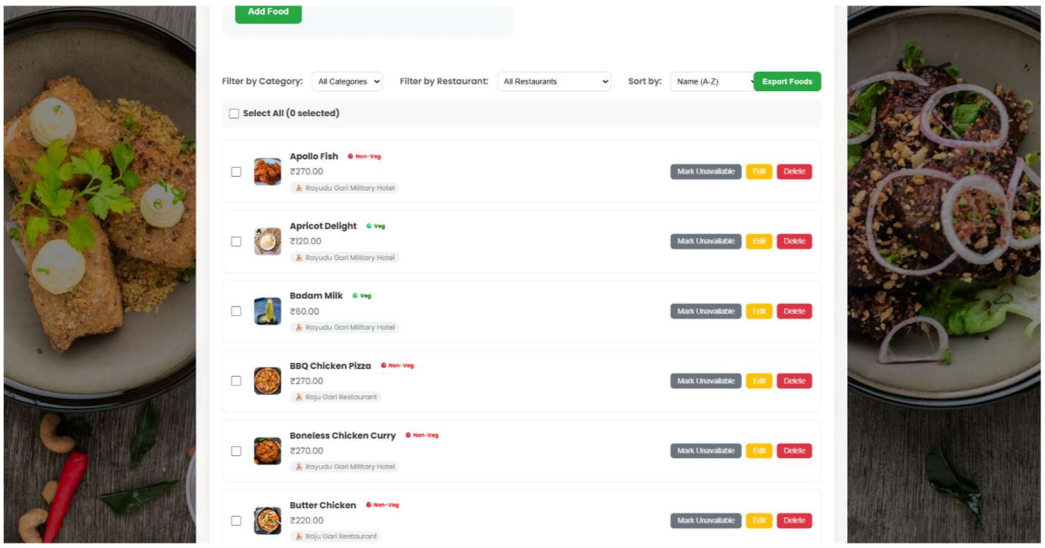

Figure 7.11: Admin Dashboard – Food List with Actions

7.12 Admin Dashboard – Partner Applications Tab

The Partner Applications tab displays all restaurant partner applications submitted through the Partner With Us form. Each row shows the Restaurant name, Owner name, Contact details, Application date, a View Doc link to the uploaded PDF, the current approval Status badge, and a dropdown to change the status to Approved or Rejected. Rejected applications trigger an automated email to the applicant.

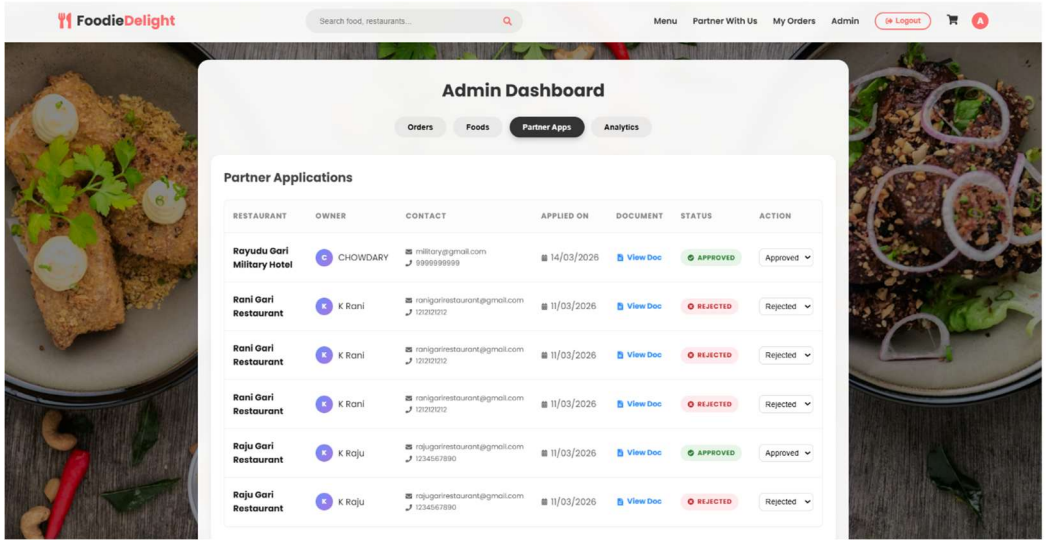

Figure 7.12: Admin Dashboard – Partner Applications Tab

7.13 Admin Dashboard – Analytics Tab

The Analytics tab provides a Global Sales Analytics overview of the entire platform. Summary cards display Total Revenue, Today's Revenue, Total Orders, and Total Users. A Revenue Trend line chart shows the revenue over the last 7 days, and an Order Status Breakdown doughnut chart visualises the proportions of Delivered, Pending, Preparing, Cancelled, and Failed orders.

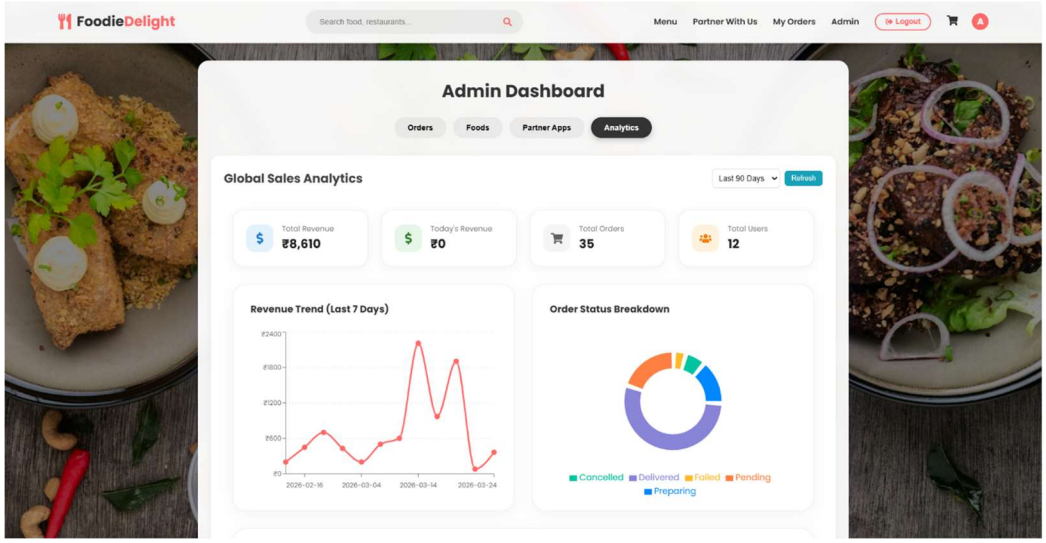

Figure 7.13: Admin Dashboard – Analytics Tab

7.14 User Profile Page

The User Profile page allows logged-in users to view and manage their personal account information. The left panel shows the user's avatar, name, and navigation links for Profile Information, Manage Addresses, and Logout. The right panel displays editable fields for Name, Gender, Email Address, Mobile Number, and a Change Password section. Options to Deactivate or Delete the account are also available at the bottom.

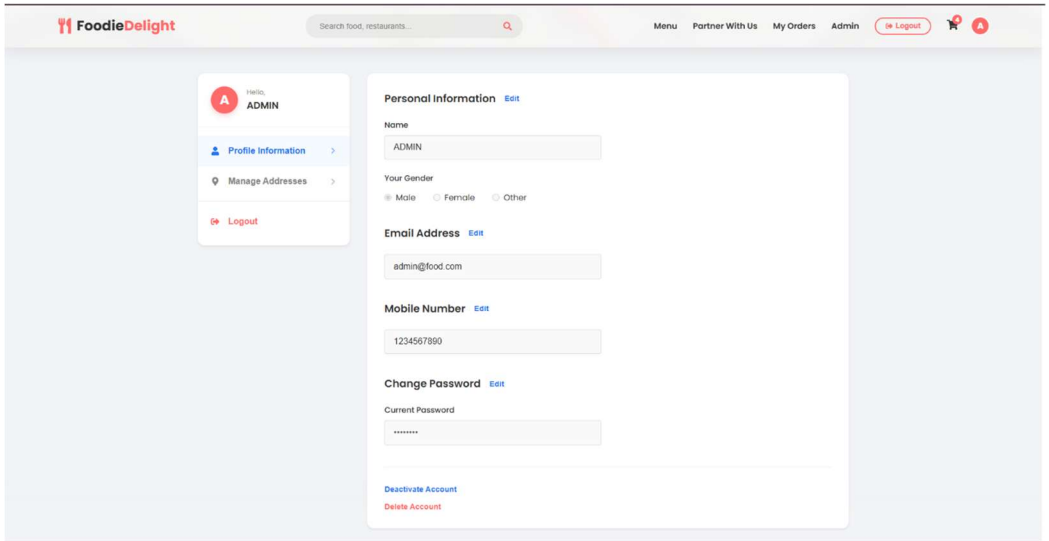

Figure 7.14: User Profile Page

CHAPTER 8
CONCLUSION

8.1 Summary

The FoodieDelight Full-Stack Food Ordering Platform has been successfully developed as a comprehensive web application using the MERN stack. The project integrates React.js with Vite for a premium glassmorphism frontend, Node.js and Express.js for a robust RESTful backend, and MongoDB with Mongoose for scalable data management across five collections.

All planned modules have been implemented and tested. The customer module provides a complete user journey from secure registration and login through menu browsing, cart management, checkout, and order tracking. The restaurant partner module delivers complete menu management with Cloudinary image uploads, smart order fulfilment, and automatic COD payment synchronisation. The admin module provides full platform oversight across four dashboard tabs: Orders, Foods, Partner Applications, and Analytics.

The project successfully demonstrates advanced full-stack development concepts including multi-role JWT authentication, Cloudinary CDN integration, SendGrid transactional emails, Framer Motion animations, Context API state management, and Vercel deployment with legacy peer dependency configuration.

8.2 Future Enhancements

While the current version of FoodieDelight is fully functional, several enhancements can be considered for future iterations of the application. These include implementing real-time order tracking using WebSockets via Socket.io to provide live status updates to customers without requiring a page refresh. Integrating a payment gateway such as Razorpay or Stripe would enable actual online payment processing instead of relying on simulation. Additionally, push notifications can be introduced using Firebase Cloud Messaging to keep users informed about order updates on their mobile devices. A customer reviews and ratings system for food items and restaurant

partners would further enhance user engagement and trust. Expanding the platform to include a delivery partner role with GPS-based route optimization and live location sharing on a map interface would improve delivery efficiency. Introducing a loyalty points and coupon code system could help with customer retention and promotional campaigns. Finally, supporting Progressive Web App features would allow users to browse offline and install the application directly on their mobile home screens.

REFERENCES

[1] MongoDB Documentation. (2024). MongoDB Manual. Retrieved from https://www.mongodb.com/docs/

[2] React Documentation. (2024). React – A JavaScript Library for Building User Interfaces. Retrieved from https://react.dev/

[3] Node.js Documentation. (2024). Node.js v24 Documentation. Retrieved from https://nodejs.org/en/docs/

[4] Express.js Documentation. (2024). Express – Fast, Unopinionated, Minimalist Web Framework. Retrieved from https://expressjs.com/

[5] Mongoose Documentation. (2024). Mongoose ODM v9. Retrieved from https://mongoosejs.com/docs/

[6] Vite Documentation. (2024). Vite – Next Generation Frontend Tooling. Retrieved from https://vitejs.dev/

[7] Cloudinary Documentation. (2024). Cloudinary – Image and Video Management. Retrieved from https://cloudinary.com/documentation

[8] Framer Motion Documentation. (2024). Framer Motion – Production-Ready Animation Library. Retrieved from https://www.framer.com/motion/

[9] JSON Web Token. (2024). JWT Introduction and Usage. Retrieved from https://jwt.io/introduction

[10] SendGrid Documentation. (2024). SendGrid Email API. Retrieved from https://docs.sendgrid.com/

APPENDIX

A.1 Backend Server Setup (server.js)

```
const express  = require('express'); const mongoose =
require('mongoose'); const cors     = require('cors'); const dotenv
= require('dotenv'); dotenv.config();  const authRoutes  =
require('./routes/auth'); const foodRoutes  =
require('./routes/foods'); const orderRoutes =
require('./routes/orders'); const adminRoutes =
require('./routes/admin');  const app = express(); app.use(cors());
app.use(express.json());  mongoose.connect(process.env.MONGO_URI)
.then(() => console.log('Connected to MongoDB'))   .catch(err  =>
console.log(err));  app.use('/api/auth',   authRoutes);
app.use('/api/foods',  foodRoutes); app.use('/api/orders',
orderRoutes); app.use('/api/admin',  adminRoutes);  const PORT =
process.env.PORT || 5000; app.listen(PORT, () => console.log(`Server
running on port ${PORT}`));
```

A.2 User Schema (models/User.js)

```
const mongoose   = require('mongoose'); const userSchema = new
mongoose.Schema({   name:       { type: String,  required: true },
email:      { type: String,  required: true, unique: true },
password:   { type: String,  required: true },   role:       { type:
String,  enum: ['user','restaurant','admin'], default: 'user' },
isVerified: { type: Boolean, default: false },   otp:        String,
otpExpiry:  Date,   resetToken: String, }, { timestamps: true });
module.exports = mongoose.model('User', userSchema);
```

A.3 Environment Configuration (.env)

```
PORT=5000 MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_super_secret_jwt_key SMTP_SERVICE=gmail
SMTP_EMAIL=your_email@gmail.com SMTP_PASSWORD=your_app_password
FROM_NAME="FoodieDelight" FROM_EMAIL=your_email@gmail.com
CLOUDINARY_CLOUD_NAME=your_cloud_name CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
```